

Estado del Arte: Técnicas de Balanceo en Series Temporales para Datos Médicos

1. Introducción

El desbalanceo de datos constituye uno de los desafíos más significativos en el aprendizaje automático aplicado a series temporales médicas. Este problema se manifiesta de múltiples formas: desbalanceo en características demográficas (sexo, edad, raza), desbalanceo en volumen de datos (diferente número de mediciones por paciente), frecuencias de muestreo irregulares, y desbalanceo de clases en eventos clínicos de interés. Esta revisión sistemática presenta un análisis exhaustivo de las técnicas disponibles para abordar estos desafíos, con especial énfasis en su aplicabilidad a datos de diabetes tipo 1.

La estructura de este documento se organiza en tres bloques principales: métodos tradicionales de remuestreo y técnicas basadas en SMOTE, métodos algorítmicos y de ensemble, y finalmente, aproximaciones modernas basadas en deep learning, incluyendo redes generativas adversarias (GANs), modelos de difusión, autoencoders variacionales (VAEs), y arquitecturas especializadas para series temporales irregulares.

2. Métodos Tradicionales de Remuestreo

2.1 Random Oversampling y Undersampling

El random oversampling constituye la técnica más básica para abordar el desbalanceo de clases, mediante la duplicación aleatoria con reemplazo de ejemplos de la clase minoritaria hasta alcanzar un ratio deseado

<https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>

. Esta aproximación presenta como principal ventaja su implementación trivial y su compatibilidad universal con cualquier algoritmo de aprendizaje, además de preservar toda la información de la clase minoritaria. Sin embargo, presenta un riesgo elevado de overfitting, dado que el modelo puede memorizar los ejemplos repetidos

<https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>

, además de incrementar significativamente el coste computacional debido al aumento del tamaño del dataset.

En contraste, el random undersampling elimina aleatoriamente ejemplos de la clase mayoritaria hasta equilibrar el dataset. Aunque reduce el coste computacional y elimina el riesgo de overfitting por repetición, esta técnica presenta el problema crítico de la pérdida de información potencialmente relevante de la clase mayoritaria, pudiendo eliminar ejemplos

cerca de la frontera de decisión que resultan particularmente informativos para el modelo.
<https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>

Una estrategia combinada que implementa un pipeline secuencial de oversampling seguido de undersampling puede balancear los beneficios y limitaciones de ambos métodos
<https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>

. Por ejemplo, en un dataset con ratio inicial 1:100 (minoritaria:mayoritaria), el oversampling podría llevar el ratio a 1:10, y posteriormente el undersampling ajustarlo a 1:2, mejorando la generalización respecto al oversampling puro mientras reduce el coste computacional.

2.2 SMOTE y sus Variantes

2.2.1 SMOTE Clásico

La técnica Synthetic Minority Over-sampling Technique (SMOTE), propuesta por Chawla et al. en 2002, representa un avance significativo sobre el random oversampling al generar ejemplos sintéticos mediante interpolación lineal en el espacio de características. El algoritmo identifica para cada muestra de la clase minoritaria sus k vecinos más cercanos (típicamente $k=5$) de la misma clase, selecciona aleatoriamente uno de ellos, y genera un nuevo ejemplo sintético mediante la fórmula $x_{syn} = x_i + \lambda \cdot (x_{nn} - x_i)$, donde λ se extrae de una distribución uniforme $U(0,1)$.

SMOTE presenta ventajas significativas al generar ejemplos sintéticos "realistas" mediante interpolación lineal, resultando más efectivo que el random oversampling para evitar overfitting, y funcionando especialmente bien con algoritmos como k-NN y SVM <https://pubmed.ncbi.nlm.nih.gov/23522326/>. Sin embargo, en espacios de alta dimensionalidad, como es común en datos médicos con más de 1000 características, SMOTE puede generar ruido y mostrar efectividad reducida con clasificadores como DLDA, DQDA o PAM <https://pubmed.ncbi.nlm.nih.gov/23522326/>. Además, puede conectar inadvertidamente outliers minoritarios con la clase mayoritaria.

Una limitación crítica de SMOTE estándar para series temporales es que no respeta las dependencias temporales inherentes a estos datos. La interpolación directa entre series temporales puede generar secuencias no realistas que no consideran autocorrelación, estacionalidad, ni dinámica temporal. Por tanto, no se recomienda aplicar SMOTE estándar directamente sobre series de datos médicos crudas, siendo preferible su aplicación sobre features agregados (media, varianza, pendiente por período) o utilizar variantes específicamente diseñadas para datos temporales.

2.2.2 Borderline-SMOTE

Han et al. (2005) propusieron Borderline-SMOTE como una mejora que focaliza la generación de sintéticos exclusivamente cerca de la frontera de decisión <https://towardsdatascience.com/class-imbalance-smote-borderline-smote-adasyn-6e36c78d804/>. El algoritmo clasifica cada muestra minoritaria según su vecindario: si todos los m vecinos más cercanos son de la clase mayoritaria, la muestra se clasifica como NOISE y se

ignora; si algunos vecinos son mayoritarios, se clasifica como BORDER y se generan sintéticos; si todos los vecinos son minoritarios, se clasifica como SAFE y se ignora.

Esta aproximación presenta ventajas al focalizar la generación donde realmente impacta la clasificación, evitando rellenar zonas ya bien representadas y reduciendo el riesgo de sobreajuste respecto a SMOTE estándar. No obstante, puede sobreenfatizar observaciones extremas, resulta sensible a outliers en la frontera, y su definición rígida de "border point" puede ser limitante.

2.2.3 ADASYN

La técnica Adaptive Synthetic Sampling (ADASYN), propuesta por He et al. en 2008, introduce generación adaptativa basada en la "dificultad de aprendizaje" de cada muestra <https://towardsdatascience.com/class-imbalance-smote-borderline-smote-adasyn-6e36c78d804/>. Para cada muestra minoritaria, ADASYN calcula un ratio de impureza $r_i = \Delta_i/k$, donde Δ_i representa el número de vecinos mayoritarios entre los k vecinos más cercanos. Este ratio se normaliza y determina el número de sintéticos a generar para cada muestra: $g_i = \hat{r}_i \cdot G$, donde G es el total de sintéticos a generar.

ADASYN presenta ventajas sobre SMOTE y Borderline-SMOTE al no emplear una clasificación binaria (noise/border/safe) sino un continuo de dificultad, generando más sintéticos para observaciones "difíciles" rodeadas de ejemplos mayoritarios, y siendo menos sensible a definiciones rígidas de frontera. Sin embargo, la comparación empírica en la literatura muestra resultados mixtos <https://www.diva-portal.org/smash/get/diva2:1519153/FULLTEXT01.pdf>: mientras He et al. (2008) reportaron superioridad de ADASYN sobre SMOTE en Decision Trees, Brandt (2021) <https://www.diva-portal.org/smash/get/diva2:1519153/FULLTEXT01.pdf> encontró que SMOTE superaba a ADASYN en SVM, Naïve Bayes y k-NN, sugiriendo que la efectividad depende significativamente del clasificador y el dataset específicos.

2.3 Métodos SMOTE Específicos para Series Temporales

2.3.1 T-SMOTE

Zhao et al. (2022) propusieron T-SMOTE (Temporal-oriented SMOTE) como solución al problema fundamental de que SMOTE estándar trata series temporales como vectores independientes sin considerar orden temporal, autocorrelación ni patrones secuenciales <https://www.ijcai.org/proceedings/2022/0334.pdf>. T-SMOTE incorpora tres innovaciones clave: primero, identifica series minoritarias "difíciles" cerca de la frontera de clase usando métricas temporales como Dynamic Time Warping (DTW) en lugar de distancia euclidiana; segundo, al interpolar preserva patrones temporales incluyendo tendencias y periodicidades; tercero, implementa weighted sampling que prioriza sintéticos generados cerca de la frontera.

Los resultados experimentales demostraron que T-SMOTE supera consistentemente a SMOTE tradicional, Random Oversampling, INOS y MBO en métricas AUC, F1 y AUPRC, siendo especialmente efectivo en predicción temprana (early prediction) <https://www.ijcai.org/proceedings/2022/0334.pdf> y funcionando tanto en datasets

univariados como multivariados. La implementación de referencia está disponible en el repositorio de Microsoft Research <https://www.ijcai.org/proceedings/2022/0334.pdf>.

2.3.2 tSMOTE (Time Sliced SMOTE)

El método Time Sliced SMOTE aborda específicamente el problema de series temporales con observaciones completamente faltantes en ciertos intervalos y longitudes de serie muy variables entre pacientes <https://arxiv.org/abs/2201.05634>. El algoritmo particiona el intervalo de interés $[t_{\min}, t_{\max}]$ en N "time slices" de tamaño $\Delta t = (t_{\max} - t_{\min})/N$, asigna cada observación al slice correspondiente, y aplica SMOTE independientemente dentro de cada slice.

Esta técnica permite imputar observaciones completamente faltantes manteniendo la estructura temporal, preserva correlaciones dentro de cada slice, y resulta compatible con SMOTE estándar facilitando su implementación. Entre sus limitaciones se encuentra la necesidad de definir manualmente el número de slices N , la pérdida de correlaciones entre slices lejanos, y el supuesto de estacionariedad local dentro de cada slice.

3. Métodos Algorítmicos: Cost-Sensitive Learning

3.1 Class Weights (Ponderación de Clases)

La ponderación de clases modifica la función de pérdida para penalizar más los errores en la clase minoritaria. Matemáticamente, la función de pérdida ponderada para clasificación binaria se expresa como $L_{\text{weighted}} = w_0 \sum_{(i \in C_0)} \ell(y_i, \hat{y}_i) + w_1 \sum_{(i \in C_1)} \ell(y_i, \hat{y}_i)$, donde w_0 y w_1 son los pesos para las clases mayoritaria y minoritaria respectivamente, y ℓ representa la pérdida base. Una fórmula comúnmente utilizada para calcular pesos balanceados es $w_c = n_{\text{samples}} / (n_{\text{classes}} \cdot n_{\text{samples}_c})$.

La efectividad de class weights varía significativamente según el algoritmo empleado. En Logistic Regression, la mejora es dramática, con reportes de aumento de recall del 0% al 57.5% al emplear `class_weight='balanced'`. En contraste, Random Forest muestra mejora mínima o nula con esta técnica (Controlling Class Weights For Imbalanced Classification. YouTube/Machine Learning Tutorial (2021))., sugiriendo que los árboles de decisión ya capturan las minorías mediante split condicional. En Deep Learning con arquitecturas LSTM/GRU, class weights se implementan mediante funciones de pérdida personalizadas <https://www.omdena.com/blog/time-series-classification-model-tutorial>.

Se recomienda probar siempre class weights antes de recurrir a remuestreo, dado que no modifica los datos y permite entrenamiento rápido, siendo especialmente efectivo en modelos lineales y redes neuronales.

3.2 Focal Loss

Propuesta por Lin et al. (2017) en el contexto de detección de objetos densos con RetinaNet, Focal Loss aborda el problema de que cross-entropy estándar penaliza igualmente ejemplos fáciles y difíciles. En datasets desbalanceados, la clase mayoritaria domina la pérdida debido a la abundancia de ejemplos fáciles

<https://pmc.ncbi.nlm.nih.gov/articles/PMC8718001/>. Focal Loss reduce automáticamente el peso de ejemplos bien clasificados mediante la fórmula $FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$, donde p_t representa la probabilidad de la clase verdadera, α_t es un peso de clase (balanceo estático), y $\gamma \geq 0$ es el focusing parameter (típicamente $\gamma=2$).

El término $(1-p_t)^\gamma$ actúa como modulador: cuando un ejemplo está bien clasificado ($p_t \rightarrow 1$), la pérdida se reduce drásticamente ($(1-p_t)^\gamma \rightarrow 0$); cuando está mal clasificado ($p_t \rightarrow 0$), la pérdida permanece alta. Con $\gamma=2$, un ejemplo con confianza 0.9 experimenta una reducción del 99% en la pérdida respecto a cross-entropy estándar, mientras que uno con confianza 0.5 solo se reduce un 75%.

Variantes como Cyclical Focal Loss (CFL) varían γ cíclicamente durante entrenamiento <https://arxiv.org/abs/2202.08978>, comenzando con γ bajo en epochs iniciales (aprendiendo de ejemplos fáciles) y aumentándolo en epochs finales (focalizando en difíciles), logrando mejora consistente sobre Focal Loss estático en benchmarks como CIFAR-10/100 <https://arxiv.org/abs/2202.08978>.

En aplicaciones médicas, Focal Loss ha demostrado efectividad significativa. En clasificación de radiografías de tórax (CXR) para neumonía bacteriana/viral/COVID-19, alcanzó 97.2% accuracy versus 92% con cross-entropy <https://pmc.ncbi.nlm.nih.gov/articles/PMC8718001/>. En series temporales, la combinación CNN-GRU con Focal Loss para detección de fallos mostró mejora especialmente notable en recall de clase minoritaria <https://www.sciencedirect.com/science/article/abs/pii/S0957417423026945>.

3.3 Cost-Sensitive Hybrid Networks

Wang et al. (2024) propusieron CSHN (Cost-Sensitive Hybrid Network) específicamente para detección de anomalías en series temporales desbalanceadas <https://www.sciencedirect.com/science/article/abs/pii/S0957417423026945>.

La arquitectura combina CNN para capturar features locales (patrones cortos) con GRU para dependencias temporales largas, procesando ambas en paralelo antes de fusionarse. La función de pérdida cost-sensitive se expresa como $L_{CS} = \sum_i \sum_c w_c \cdot \mathbb{1}[y_i=c] \cdot \ell(f(x_i), c)$, donde w_c representa el peso (coste) de misclassification para la clase c , asignando mayor w_c para clases minoritarias.

Los experimentos en datasets industriales de velocidad y temperatura demostraron mejoras de 3-5% en G-means y 5-9% en F-measure versus cross-entropy <https://www.sciencedirect.com/science/article/abs/pii/S0957417423026945>

, siendo especialmente efectivo en ratios de desbalance de 1:10 a 1:50.

Complementariamente, Roychoudhury et al. (2017) propusieron Cost-Sensitive Shapelets https://www.researchgate.net/publication/320564096_Cost_Sensitive_Time-series_Classification, una aproximación altamente interpretable que aprende conjuntamente shapelets (subsecuencias cortas características de una clase), sus pesos, y los costes de

misclassification. La técnica resulta competitiva con SMOTE y undersampling en 18 datasets desbalanceados

https://www.researchgate.net/publication/320564096_Cost_Sensitive_Time-series_Classification, con la ventaja adicional de que los shapelets son patrones visualizables. Sin embargo, presenta limitaciones de escalabilidad debido a que la búsqueda de shapelets tiene complejidad $O(n^2m^2)$, donde m es la longitud de la serie.

4. Métodos Ensemble para Datos Desbalanceados

4.1 EasyEnsemble

Liu et al. (2009) propusieron EasyEnsemble como método que explota toda la información de la clase mayoritaria, superando las limitaciones del undersampling simple <https://sci2s.ugr.es/keel/pdf/algorithm/articulo/2009-IEEE%20TSMCpartB%20Exploratory%20Undersampling%20for%20Class%20Imbalance%20Learning.pdf>. El algoritmo undersamplea T subconjuntos balanceados de tamaño n (aproximadamente igual al tamaño de la clase minoritaria) de la clase mayoritaria (tamaño N), combina cada subconjunto con toda la clase minoritaria para crear datasets balanceados D_i , entrena un clasificador base (típicamente AdaBoost con C4.5) en cada D_i , y finalmente agrega las predicciones mediante votación o promediado.

Las ventajas principales incluyen la explotación de toda la información de la clase mayoritaria (versus undersampling simple que descarta datos), métricas superiores de AUC, F-measure y G-mean respecto a métodos individuales <https://tungmphung.com/imbalanced-learning-sampling-techniques/>

, y un efecto "bagging" que mejora la robustez. La principal limitación es el requisito de entrenar T modelos, incrementando el coste computacional, aunque el beneficio marginal decrece si T es muy grande.

4.2 BalanceCascade

BalanceCascade se diferencia de EasyEnsemble por su eliminación progresiva de ejemplos mayoritarios bien clasificados <https://pmc.ncbi.nlm.nih.gov/articles/PMC5914103/>. En cada iteración t , el algoritmo undersamplea un subconjunto S_t de la clase mayoritaria del pool actual, entrena un clasificador M_t en S_t más la clase minoritaria, y elimina del pool mayoritario los ejemplos correctamente clasificados con alta confianza. Este proceso se repite hasta alcanzar T clasificadores.

La ventaja sobre EasyEnsemble radica en su mayor eficiencia en datasets muy desbalanceados (ratios superiores a 1:100) <https://tungmphung.com/imbalanced-learning-sampling-techniques/>, focalizando clasificadores posteriores en ejemplos "difíciles" de la clase mayoritaria. Sin embargo, puede eliminar prematuramente ejemplos de frontera y resulta sensible al orden de eliminación, pudiendo ser subóptimo.

4.3 BEBS (Bagging of Extrapolation Borderline-SMOTE SVM)

Wang et al. (2017) propusieron BEBS como método que combina tres técnicas exitosas: Borderline-SMOTE para generar sintéticos en la frontera, Bagging para diversidad de modelos, y SVM como clasificador potente con margen máximo <https://pmc.ncbi.nlm.nih.gov/articles/PMC5304315/>. La arquitectura identifica border points con Borderline-SMOTE, genera K subconjuntos bootstrap, aplica Borderline-SMOTE en cada subconjunto para crear datasets balanceados, entrena SVM en cada uno, y agrega mediante votación mayoritaria.

Un hallazgo notable es su insensibilidad a hiperparámetros en el rango para K (número de metaclasificadores) <https://pmc.ncbi.nlm.nih.gov/articles/PMC5304315/>, manteniendo G-means estable con variación máxima de ~ 0.15 , y logrando buen balance entre recall y precision.

5. Threshold Moving (Ajuste de Umbral de Decisión)

La mayoría de clasificadores generan probabilidades $p(y=1|x)$ que se convierten a clase mediante un umbral τ (típicamente 0.5) <https://machinelearningmastery.com/threshold-moving-for-imbalanced-classification/>. En datos desbalanceados, este umbral asume clases equiprobables a priori, lo cual es subóptimo en datasets con ratios extremos como 1:99. Threshold Moving ajusta τ para optimizar métricas de interés (F1, G-mean, etc.).

Las ventajas principales son que no requiere reentrenamiento ni remuestreo de datos, su implementación rápida como post-procesamiento, y su compatibilidad con cualquier clasificador que genere probabilidades. La limitación es que no mejora la capacidad discriminativa del modelo, solo ajusta la decisión final, y requiere un conjunto de validación para optimizar τ .

Métodos de optimización incluyen: Youden's J Statistic ($J = \text{TPR} - \text{FPR}$), que encuentra $\tau^* = \arg \max_{\tau} J(\tau)$ usando la curva ROC; optimización basada en Precision-Recall Curve maximizando F1-score; y Grid Search sobre métricas custom como $G\text{-mean} = \sqrt{(\text{Sensitivity} \cdot \text{Specificity})}$.

Riniker et al. (2021) propusieron GHOST (Gradient-based Hyperparameter Optimization for Sequential Thresholding) como procedimiento automatizado para seleccionar el threshold óptimo en Random Forest, demostrando que supera random undersampling en 138 datasets de drug discovery sin requerir reentrenamiento ni remuestreo <https://pubs.acs.org/doi/10.1021/acs.jcim.1c00160>.

6. Data Augmentation para Series Temporales

6.1 Survey Empírico Fundamental

Iwana y Uchida (2021) realizaron un survey empírico exhaustivo evaluando 12 técnicas de augmentation en 128 datasets del UCR Time Series Archive con redes neuronales (FCN, ResNet, LSTM) <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>. Los hallazgos principales revelaron que Time Warping y Magnitude Warping son las mejores

técnicas en promedio

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>, Window Slicing resulta muy efectivo para aumentar volumen de datos, Jittering es simple pero sorprendentemente efectivo, y las combinaciones de Time Warping con Magnitude Warping superan cualquier técnica individual.

6.2 Técnicas Específicas

6.2.1 Time Warping

Time Warping estira o comprime el eje temporal de manera no uniforme, preservando la forma general de la serie

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>. El algoritmo genera una curva de warping suave $\tau(t) = t + \sigma \cdot \sin(2\pi ft + \phi)$, donde σ es la magnitud de deformación, f la frecuencia de oscilación, y ϕ la fase aleatoria, para luego resamplear la serie original en los nuevos tiempos: $x'(t) = x(\tau(t))$.

La justificación biológica en diabetes es clara: las curvas de glucosa postprandial tienen forma similar (subida-pico-bajada), pero velocidad variable según el índice glucémico del alimento, cantidad de insulina activa, y actividad física previa. Time Warping simula esta variabilidad natural.

6.2.2 Magnitude Warping

Magnitude Warping multiplica la serie por una curva suave aleatoria, simulando cambios en magnitud sin alterar timing

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>. El algoritmo genera una curva de scaling suave $s(t) = 1 + \sigma \cdot \text{spline}(t)$, donde $\text{spline}(t)$ es una B-spline con puntos de control aleatorios, y aplica $x'(t) = s(t) \cdot x(t)$.

Esta técnica simula la variabilidad de sensibilidad a insulina entre días en diabetes: un día con alta sensibilidad produce mayor caída de glucosa con el mismo bolo, mientras que un día con resistencia produce menor caída. Magnitude Warping escala la amplitud de la respuesta glucémica.

6.2.3 Window Slicing

Window Slicing corta series largas en múltiples ventanas más cortas, multiplicando el número de muestras

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>. Las estrategias incluyen sliding window (ventanas solapadas) y non-overlapping (ventanas consecutivas sin solapamiento).

Las ventajas son que multiplica el dataset sin generar sintéticos (solo reorganiza datos reales) y permite igualar volumen entre pacientes con distinta duración de seguimiento. Las limitaciones incluyen pérdida de contexto de largo plazo (una ventana de 24h no captura tendencias semanales) y posible introducción de correlación artificial entre muestras (ventanas solapadas del mismo paciente).

6.2.4 Jittering

Jittering añade ruido gaussiano pequeño: $x'(t) = x(t) + \varepsilon$, donde $\varepsilon \sim N(0, \sigma^2)$ <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254841>. Es extremadamente simple, proporciona regularización efectiva previniendo overfitting, y resulta compatible con cualquier tipo de serie. La limitación es que puede generar valores irreales si σ es muy grande y no respeta constraints físicos.

7. Modelos Generativos: GANs

7.1 RGAN/RCGAN: Baseline Clásico

Esteban et al. (2017) propusieron RGAN/RCGAN como primera aplicación sistemática de GANs a series temporales médicas multivariadas <https://arxiv.org/abs/1706.02633>. RGAN procesa secuencialmente ruido $z_t \sim N(0, I)$ mediante LSTM para generar series sintéticas $\{\hat{x}_1, \dots, \hat{x}_T\}$, mientras que RCGAN añade condicionamiento concatenando la condición c (edad, sexo) en cada timestep. El discriminador evalúa la probabilidad real/fake por timestep, no solo al final.

La función de pérdida incorpora Maximum Mean Discrepancy: $L_G = L_{GAN} + \lambda \cdot \text{MMD}(P_{\text{real}}, P_{\text{gen}})$, donde MMD mide discrepancia entre distribuciones de series reales y generadas.

Los experimentos en datasets ICU (eICU/MIMIC-III) con 17,000 pacientes y series de presión arterial, frecuencia cardíaca, SpO2 y temperatura evaluaron mediante Train on Synthetic, Test on Real (TSTR), logrando accuracy solo 2-3% menor que Train on Real, Test on Real <https://arxiv.org/abs/1706.02633>. El test estadístico MMD two-sample arrojó p-values de 0.09-0.38, no permitiendo rechazar la hipótesis de que ambas distribuciones son iguales <https://arxiv.org/abs/1706.02633>. El código de referencia está disponible en el repositorio de ratschlab <https://github.com/ratschlab/RGAN>.

7.2 DoppelGANger: Estado del Arte

Lin et al. (2020) propusieron DoppelGANger como respuesta a que RGAN/RCGAN no capturan adecuadamente correlaciones temporales a múltiples escalas (diarias, semanales, estacionales) y experimentan modo collapse frecuente en series largas <https://arxiv.org/pdf/1909.13403>. Las innovaciones clave incluyen: batch generation que genera bloques de S timesteps simultáneamente preservando correlaciones de medio/largo plazo <https://www.gretel.ai/blog/create-synthetic-time-series-with-doppelganger-and-pytorch>; auto-normalization donde cada serie se normaliza individualmente y los valores $\min(x)$ y $\max(x)$ se tratan como metadata que la GAN también aprende a generar, evitando modo collapse <https://arxiv.org/html/2405.20959v1>; y auxiliary discriminator que evalúa solo atributos estáticos, mejorando la generación de relaciones atributo-serie temporal <https://arxiv.org/html/2405.20959v1>.

Los componentes incluyen Attribute Generator para metadata (sexo, edad, min, max), Feature Generator para series temporales condicionadas en metadata, Discriminator que evalúa realismo de (metadata + series), y Auxiliary Discriminator que evalúa solo metadata.

Los parámetros clave son `max_sequence_len` (longitud total), `sample_len` (S: tamaño de bloque generado simultáneamente), `batch_size`, `epochs`, y `learning_rate`.

Los resultados reportados incluyen captura de tendencias semanales y anuales en Wikipedia web traffic, correlaciones día/noche en redes de comunicación, y runtime ~40x más rápido que la implementación TensorFlow 1 original. La implementación PyTorch está disponible en Gretel AI

<https://www.gretel.ai/blog/create-synthetic-time-series-with-doppelganger-and-pytorch>.

7.3 C-TimeGAN/Conditional TimeGAN

TimeGAN, propuesto por Yoon et al. (2019) en NeurIPS, combina adversarial training con supervised training para aprender representaciones temporales. Los componentes incluyen Embedding Network ($x \rightarrow h$), Recovery Network ($h \rightarrow \tilde{x}$), Generator ($z \rightarrow h_{\text{syn}}$), Discriminator (distingue h real vs h_{syn}), y Supervisor (aprende dinámica temporal $h_t \rightarrow h_{t+1}$).

C-TimeGAN extiende TimeGAN añadiendo condicionamiento explícito

https://www.researchgate.net/publication/379312776_Conditional-TimeGAN_for_Realistic_and_High-quality_Appliance_Trajectories_Generation_and_Data_Augmentation_in_Non-intrusive_Load_Monitoring. Liu et al. (2024) propusieron Conditional-TimeGAN para generación realista y de alta calidad, mientras que Zhang et al. desarrollaron Continuous Conditional GANs con regularización del generador

<https://openreview.net/pdf/a314da3379d678f622a913c79365c620d5d935e5.pdf>. La arquitectura modificada incluye Generator $G(z, c) \rightarrow h_{\text{syn}}$, Discriminator $D(h, c) \rightarrow P(\text{real}|h, c)$, y Supervisor $S(h_t, c) \rightarrow h_{t+1}$, donde c representa la condición.

MC-TimeGAN, propuesto por Demirel et al. (2025), extiende TimeGAN condicional para series multivariadas (múltiples features correlacionadas) Demirel et al. (2025).

MC-TimeGAN: Multivariate Conditional TimeGAN. GitHub., con aplicación original en congestión de redes eléctricas. El código está disponible en el repositorio KIT-IAI Demirel et al. (2025). MC-TimeGAN: Multivariate Conditional TimeGAN. GitHub..

CTGAN-MSIN (2025) combina Conditional TimeGAN con Multi-Scale Importance-aware Network para desbalanceo extremo (1:15 o mayor) A Gradient-Penalized Conditional TimeGAN Combined with Multi-Scale Importance-aware Network (2025).. Las innovaciones incluyen Gradient Penalty para estabilizar entrenamiento (similar a WGAN-GP) y Multi-Scale Importance-aware Network que atiende a patrones temporales a múltiples escalas, logrando >98% accuracy en diagnóstico tras augmentation condicional, superando a SMOTE/ADASYN A Gradient-Penalized Conditional TimeGAN Combined with Multi-Scale Importance-aware Network (2025)..

7.4 HealthGen: Equidad en Datos Médicos

Bing et al. (2022) propusieron HealthGen específicamente para equidad en datos clínicos <https://pmc.ncbi.nlm.nih.gov/articles/PMC9931259/>. Los datos clínicos de Electronic Health Records presentan desbalanceo demográfico (pocos pacientes de minorías étnicas),

patrones de missingness informativos (ausencia de medición puede indicar estabilidad o gravedad), e irregularidad temporal (visitas no programadas).

Las innovaciones de HealthGen incluyen modelado explícito de patrones de missingness (no solo valores observados) <https://pmc.ncbi.nlm.nih.gov/articles/PMC9931259/>, generación condicional en características demográficas y clínicas, y evaluación downstream que mejora fairness de modelos predictivos. La arquitectura emplea encoder-decoder con atención temporal, generando tanto valores como máscaras de observación.

Los experimentos en MIMIC-III demostraron que modelos entrenados en datos sintéticos generalizaron mejor a grupos minoritarios <https://pmc.ncbi.nlm.nih.gov/articles/PMC9931259/>, reduciendo la disparidad demográfica en 15-20%.

7.5 WGAN-GP para Series Temporales Médicas

Isasa et al. (2024) propusieron WGAN-GP con alignment loss para preservar correlaciones inter-variables <https://link.springer.com/article/10.1186/s12911-024-02427-0>. El alignment loss se define como $L_{\text{align}} = ||C_{\text{real}} - C_{\text{syn}}||_1$, donde C es la matriz de correlación entre variables. La función de pérdida del generador es $L_G = L_{\text{WGAN}} + \lambda_{\text{align}} \cdot L_{\text{align}}$, típicamente con $\lambda_{\text{align}} = 10.0$ <https://link.springer.com/article/10.1186/s12911-024-02427-0>.

Los datasets evaluados (TMET monitoreo cardíaco, Hypotension subset en UCI, PMData cuidados paliativos) mostraron TSTR de 0.837-0.900 (muy cercano a Train on Real) <https://link.springer.com/article/10.1186/s12911-024-02427-0> y Membership Inference Attack con precision = 0, indicando privacidad de datos sintéticos <https://link.springer.com/article/10.1186/s12911-024-02427-0>. Los hiperparámetros reportados incluyen 5000 epochs, batch size 32-64, 5 critic updates por generator update, learning rate 0.001, y gradient penalty coefficient 10.0.

Liu et al. (2025) propusieron HR-ACGAN (High Reliability ACGAN) combinando ACGAN (Auxiliary Classifier GAN) con ideas de WGAN-GP para estabilidad de entrenamiento y alta diferenciación de clases <https://www.mdpi.com/1099-4300/27/2/111>. ACGAN permite generar sintéticos etiquetados directamente sin necesidad de etiquetado post-generación.

8. Modelos de Difusión (Diffusion Models)

Los modelos de difusión presentan ventajas sobre GANs: entrenamiento más estable (no adversarial), mejor captura de variabilidad mediante muestreo estocástico, y constituyen el estado del arte en generación de imágenes (DALL-E 2, Stable Diffusion). Su aplicación a series temporales es emergente (2023-2025).

Fadlon et al. (2025) propusieron un modelo de difusión para generar series temporales regulares desde datos irregulares con completion y masking <https://arxiv.org/pdf/2510.06699>. El problema que resuelve incluye series temporales con muestreo irregular (timestamps no alineados) y el objetivo de generar series regulares (timestamps fijos) desde datos irregulares.

El pipeline en dos pasos incluye: primero, Time Series Transformer que completa irregularidades tomando como input la serie irregular $\{(x_{t1}, t1), (x_{t2}, t2), \dots\}$, usa transformer con positional encoding de timestamps continuos, y genera como output una serie regular imputada en timesteps fijos; segundo, Diffusion Model que genera series sintéticas tomando como input las series regulares del Paso 1, aplicando forward diffusion añadiendo ruido progresivamente, y reverse diffusion aprendiendo a des-noisear.

Las ventajas incluyen manejo nativo de irregularidad sin requerir pre-procesamiento manual y generación de sintéticos de alta calidad preservando autocorrelación y estacionalidad.

TimeGrad de Rasul et al. (ICML 2021) y SSSD de Alcaraz y Strodthoff (2022) se orientan principalmente a forecasting e imputación, no tanto a generación incondicional, con la limitación de no permitir condicionamiento explícito en características demográficas.

9. Variational Autoencoders (VAEs)

9.1 VGAE para EHRs

Nikolentzos et al. (2023) propusieron el uso de Variational Graph Autoencoders para generar registros electrónicos de salud sintéticos, representando trayectorias de pacientes como grafos dirigidos acíclicos (DAG), no como secuencias <https://www.nature.com/articles/s41746-023-00822-x>. La arquitectura incluye encoder que mapea el grafo del paciente G a distribución latente $N(\mu, \Sigma)$ y decoder que reconstruye el grafo desde muestra latente $z \sim N(\mu, \Sigma)$.

Las ventajas incluyen captura de correlaciones de largo plazo mediante arcos entre eventos lejanos en el tiempo, y generación de trayectorias realistas y privadas con MMD = 0.025 entre reales y sintéticas <https://www.nature.com/articles/s41746-023-00822-x>. La evaluación mostró que clasificadores entrenados en sintéticos alcanzaron 90-91% accuracy en datos reales, casi igual que Train on Real <https://www.nature.com/articles/s41746-023-00822-x>.

9.2 VAE Framework para Healthcare

Un framework VAE para generación de datos sintéticos healthcare (IJAIA 2025) emplea un pipeline con datos origen de Synthea (generador de EHRs sintéticos baseline), entrenamiento mediante VAE con encoder-decoder fully-connected, y evaluación mediante Jensen-Shannon divergence entre real y sintético <https://aircconline.com/ijaia/V16N3/16325ijaia02.pdf>.

Los resultados mostraron JS divergence de 0.24-0.53 (aceptable) y performance de modelos entrenados en sintéticos 2-3% menor que en reales <https://aircconline.com/ijaia/V16N3/16325ijaia02.pdf>, concluyendo que los sintéticos son válidos para pre-training y desarrollo de modelos.

Un Multitask VAE para forecasting de glucosa con manejo de missing data <https://arxiv.org/abs/2410.00015> ofrece capacidades de imputación (rellenar huecos en

series de CGM) y generación (crear perfiles de glucosa sintéticos), aunque los detalles arquitecturales son limitados al ser un paper reciente.

10. Arquitecturas para Series de Longitud/Frecuencia Variable

10.1 Variable-Length LSTM/GRU

Bashiri et al. (2024) desarrollaron modelos de predicción clínica mediante deep learning usando series temporales de longitud variable <https://pubmed.ncbi.nlm.nih.gov/38679906/>. El problema que abordan son las historias clínicas (EHR) con longitudes muy variables (desde días hasta años).

La solución mediante Piece-wise Linear Encoding (PLE) divide la serie en segmentos, ajusta una recta en cada segmento (pendiente + intercepto), y alimenta el LSTM con (pendiente, intercepto, duración) en lugar de valores crudos. Las ventajas incluyen compresión de información reduciendo dimensionalidad, procesamiento por LSTM de "resúmenes" de segmentos en lugar de puntos individuales, y funcionamiento con series desde 10 hasta 10,000 timesteps.

Los resultados mostraron mejora en performance en EHRs de longitud variable versus LSTM directo <https://pubmed.ncbi.nlm.nih.gov/38679906/>.

10.2 CoFormer (Compatible Transformer)

Wei et al. (2023) propusieron CoFormer (Compatible Transformer) para series temporales multivariadas muestreadas irregularmente <https://arxiv.org/abs/2310.11022>. El problema que resuelve incluye series con timestamps y variables no alineadas (misalignment temporal y de features).

La solución incorpora variate-time embeddings donde cada medición ($x_i(t_i)$, t_i , $feature_i$) se embebe como vector único, timestamp encoding continuo usando funciones sinusoidales para timestamps continuos (no discretos), y transformer attention que atiende entre todas las mediciones sin requerir alineamiento.

La ventaja es que no necesita imputación previa, manejando irregularidad nativamente <https://arxiv.org/abs/2310.11022>.

10.3 ViTST (Vision Transformer for Irregular Time Series)

Li et al. (2023) propusieron ViTST en NeurIPS 2023, convirtiendo series temporales en imágenes (line graphs) y usando Vision Transformer pre-entrenado <https://arxiv.org/abs/2303.12799>. El pipeline renderiza la serie temporal como gráfico de línea en imagen 224×224, pasa la imagen a Vision Transformer (ej. ViT-Base pre-entrenado en ImageNet), y fine-tunea para clasificación de series temporales.

Las ventajas incluyen aprovechamiento de pre-training de ViT (transferencia desde visión a tiempo), manejo de irregularidad (gráfico puede tener puntos no uniformemente espaciados), y strong results versus SoTA especializado <https://arxiv.org/abs/2303.12799>. La limitación es la falta de interpretabilidad (difícil explicar qué patrón temporal aprendió el modelo). El código está disponible en el repositorio Leezekun/ViTST <https://github.com/Leezekun/ViTST>.

10.4 DSTTN (Dense Spatio-Temporal Transformer)

Qian et al. (2024) propusieron DSTTN para imputación de series temporales con datos faltantes mediante redes transformer densas espacio-temporales <https://www.aimspress.com/article/doi/10.3934/mbe.2024220?viewType=HTML>. El problema específico es Complete Data Missing (CDM), series con observaciones completamente faltantes (no solo valores aislados).

Las innovaciones incluyen cross-modal imputation usando datos de modalidad espacial (ubicación geográfica, características estáticas) para imputar series temporales faltantes, dense connections con skip connections entre bloques transformer (estilo DenseNet), y Graph Laplacian regularization que preserva estructura de grafo entre sensores/pacientes.

11. Neural ODEs y Continuous-Time Models

11.1 Latent ODEs

Rubanova et al. (NeurIPS 2019) <https://arxiv.org/pdf/1907.03907> propusieron Latent ODEs para series temporales muestreadas irregularmente. En lugar de modelar series como secuencias discretas $\{x_1, x_2, \dots, x_T\}$, modelan como dinámica continua $dx/dt = f(x, t)$.

La arquitectura incluye encoder VAE que mapea observaciones irregulares a distribución latente z_0 , ODE Solver que integra $dz/dt = f_\theta(z, t)$ para obtener $z(t)$ en cualquier t , y decoder que mapea $z(t)$ de vuelta a espacio observable $x(t)$.

Las ventajas incluyen unificación de pacientes con distintas frecuencias de muestreo en un mismo modelo continuo, capacidad de consultar valor en cualquier timestamp (interpolación perfecta), y elegancia teórica superior a métodos discretos. Las limitaciones incluyen entrenamiento más complejo requiriendo ODE solvers diferenciables y mayor lentitud que RNNs debido a integración numérica.

11.2 Neural CDEs

Neural Controlled Differential Equations (CDEs) extienden ODEs incorporando señales de control (inputs externos) en la dinámica mediante la ecuación $dz/dt = f(z, t) \cdot dx/dt$. La aplicación es relevante cuando hay inputs externos irregulares (insulina, comidas) que afectan la dinámica del sistema.

12. Comparación y Recomendaciones

La comparación entre modelos GAN muestra que RGAN no ofrece condicionamiento ni manejo de irregularidad/missingness/desbalanceo; RCGAN añade condicionamiento básico; TimeGAN introduce dinámicas temporales pero sin condicionamiento; C-TimeGAN proporciona condicionamiento avanzado y manejo de desbalanceo; MC-TimeGAN extiende a series multivariadas; DoppelGANger maneja longitud variable con metadata; HealthGen ofrece la solución más completa para irregularidad, missingness y desbalanceo con foco en equidad; WGAN-GP proporciona estabilidad pero depende de implementación para condicionamiento; y CTGAN-MSIN está específicamente diseñado para desbalanceo extremo.

Las técnicas más prometedoras por tipo de desbalanceo son: para desbalanceo demográfico (sexo, edad, raza), los generativos condicionales C-TimeGAN, HealthGen y DoppelGANger generan sintéticos de subgrupos específicos manteniendo realismo; para desbalanceo de volumen (pacientes con pocos datos), data augmentation mediante Time Warping, Magnitude Warping y Window Slicing, junto con arquitecturas adaptativas como Variable-length LSTM con PLE, multiplican datos sin generación sintética reduciendo riesgo de artefactos; para desbalanceo de frecuencia (sensores irregulares), imputación avanzada mediante DSTTN y CoFormer, modelos continuos como Neural ODEs, y Transformers con Time Series Transformer + Diffusion manejan irregularidad nativamente sin descartar información; para desbalanceo de clase (eventos raros), métodos algorítmicos como Focal Loss y Cost-Sensitive Hybrid Networks, ensemble mediante EasyEnsemble y BalanceCascade, generativos como T-SMOTE y C-TimeGAN para clase minoritaria, y post-procesamiento mediante Threshold Moving ofrecen soluciones donde Focal Loss y Class Weights no requieren modificar datos permitiendo pruebas rápidas.